UCC UNIVERSIDAD CATÓLICA DE CÓRDOBA JESUITAS

Calidad de Software

Mag. Pablo Virgolini

28 de septiembre de 2026

¿Avanzamos?



¿Cómo seleccionamos una herramienta en el ecosistema de testing ?

Paso 1 y 2: requisitos e investigación de mercado

1 · Definición de requisitos

- ¿Qué problema concreto queremos resolver?
- ¿Quién la va a usar y con qué frecuencia?
- ¿Con qué otras herramientas tiene que integrarse?
- ¿Qué presupuesto y qué plazo son razonables?
- ¿Qué pasa si no la implementamos?

2 · Investigación de mercado

- Relevar opciones que cumplan los requisitos.
- Comparar: funcionalidad, costo, soporte, comunidad.
- Leer reseñas y hablar con otros equipos que ya las usan.
- Armar una lista corta (shortlist) de 2 o 3 candidatas.

Sin requisitos claros, cualquier herramienta "parece" buena: el problema hay que definirlo antes de mirar soluciones.

Paso 3: prueba de concepto (PoC)

Un piloto acotado, con datos y personas reales, antes de comprometerse con la compra o el despliegue total.

1. **Alcance acotado**
   Un equipo, un proyecto o un módulo, no toda la organización.

2. **Criterios de éxito definidos antes**
   ¿Qué tiene que demostrar la herramienta para que la aprobemos?

3. **Tiempo limitado**
   Días o pocas semanas: no es una implementación disfrazada de prueba.

4. **Con el equipo que la va a usar**
   Feedback de quienes trabajan día a día, no solo de quien decide comprarla.

Preguntas que responde la PoC

- ¿Cumple lo que prometía el proveedor?
- ¿Se integra con lo que ya usamos?
- ¿El equipo la puede aprender en un tiempo razonable?
- ¿El costo real (TCO) sigue siendo conveniente?

La PoC es la manera concreta de contrastar las expectativas del bloque anterior con la realidad, antes de firmar un contrato largo.

Paso 4 y 5: evaluación objetiva y despliegue incremental

4 · Evaluación objetiva

Una matriz de decisión pondera cada criterio y compara candidatas con números, no con preferencias.

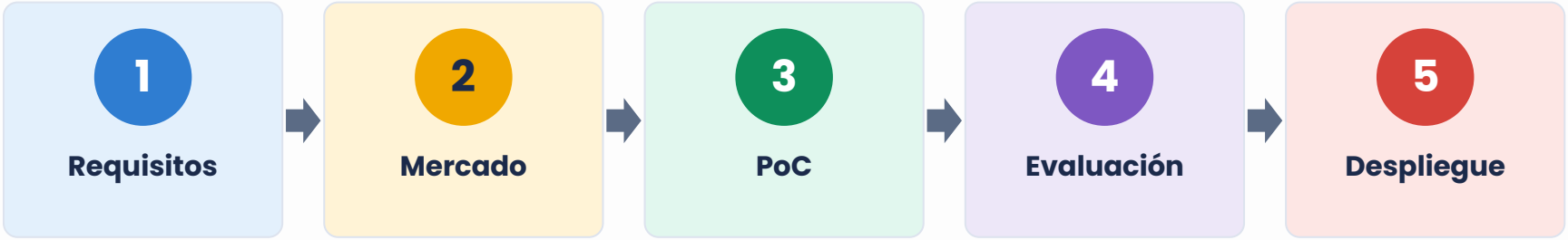
Criterio	Peso	Puntaje
Funcionalidad	30%	8
Costo (TCO)	25%	6
Facilidad de uso	20%	9
Soporte	15%	7
Integración	10%	8

5 · Despliegue incremental

1. **Piloto**
   El equipo de la PoC
2. **Expansión**
   Algunos equipos más
3. **Organización**
   Todos, con soporte interno ya rodado

Ir de a poco permite corregir el rumbo (o abandonar) con un costo mucho menor que un "big bang" a toda la organización.

El proceso completo, de punta a punta



¿Y si en cualquier paso algo no cierra?

Se vuelve atrás. Si la PoC no cumple los criterios de éxito, no se avanza a evaluación. Si la evaluación da un resultado ajustado, se prueba con otra candidata antes de desplegar. El proceso es iterativo, no una línea recta obligatoria.

El objetivo del proceso es reducir los riesgos del bloque 3, no agregar burocracia por sí misma.

JIRA: gestión de proyectos e issues

JIRA es una herramienta de seguimiento de trabajo (issues): tickets que representan tareas, historias de usuario, bugs o casos de prueba.

Un issue de JIRA tiene...

- Tipo: historia, tarea, bug, caso de prueba (con Zephyr/Xray).
- Estado: To Do → In Progress → In Review → Done.
- Responsable, prioridad y etiquetas.
- Comentarios, adjuntos e historial de cambios.
- Enlaces a otros issues (bloquea, se relaciona con, es duplicado de).

Zephyr / Xray sobre JIRA

- Agregan el tipo de issue "Caso de prueba".
- Permiten armar ciclos de ejecución dentro del mismo proyecto.
- Cada ejecución puede generar un bug vinculado automáticamente.
- La trazabilidad (bloque 2) queda dentro de la misma herramienta que usa desarrollo.

Ventaja clave: testing, desarrollo y gestión de proyecto comparten una sola fuente de verdad.

Panorama general del ecosistema

Ninguna herramienta lo hace todo: cada una resuelve una parte del ciclo de pruebas. El truco es saber cuál corresponde a cada problema.

Gestión de pruebas

Planificar, organizar y trazar

Análisis estático

Revisar el código sin ejecutarlo

Diseño / BDD

Especificar el comportamiento

Ejecución / registro

Automatizar y correr pruebas

Rendimiento

Medir carga, estrés y escala

Pruebas específicas

Redes, protocolos y nichos

Análisis estático: SonarQube

Analiza el código sin ejecutarlo: busca problemas de calidad, mantenibilidad y seguridad antes de que lleguen a producción.

Bugs

Errores probables en tiempo de ejecución.

Vulnerabilidades

Puntos débiles de seguridad conocidos.

Code smells

Código que funciona pero es difícil de mantener.

Duplicación

Bloques de código repetidos innecesariamente.

Cobertura

Porcentaje del código cubierto por pruebas.

Quality Gate

Un umbral de calidad: si el código no lo supera (por ejemplo, más bugs críticos de los permitidos), el pipeline de CI/CD se detiene y no avanza.

Se integra en CI/CD: revisa cada commit y complementa a las pruebas dinámicas, no las reemplaza.

Ejecución y registro: automatización multiplataforma

Ejecutan las pruebas y registran el resultado. Cada una cubre una plataforma o un nivel distinto.



Selenium / Cypress

Web: automatizan el navegador (visto en Unidad 10).



Appium

Aplicaciones móviles nativas, híbridas y web, en Android e iOS.



JUnit

Runner de pruebas unitarias e integración en Java: la base de la pirámide.

Registro de la ejecución

Todas dejan evidencia: logs, capturas, reportes en JUnit XML o HTML. Ese registro es lo que después consumen las herramientas de gestión (TestRail, Xray) para mostrar el estado real de la suite.

Esta categoría es la que más se trabajó en la Unidad 10: Selenium, Cypress y sus patrones (POM, TDD).

Pruebas de rendimiento

Miden cómo se comporta el sistema bajo carga: velocidad, estabilidad y capacidad máxima antes de fallar.

Herramienta	Modelo	Qué aporta
JMeter	Open source	Genera carga simulando muchos usuarios concurrentes; muy extendido y gratuito.
Gatling	Open source	Scripts como código (Scala/Kotlin); reportes detallados y buen rendimiento del propio generador de carga.
BlazeMeter	Comercial / cloud	Plataforma en la nube compatible con JMeter, con carga masiva distribuida e informes gerenciales.

Carga

Comportamiento con el tráfico esperado.

Estrés

Punto de quiebre: cuánto soporta hasta fallar.

Escalabilidad

Cómo responde al crecer usuarios o datos.

Pruebas específicas: redes y telecomunicaciones

Herramientas de nicho para dominios con protocolos y capas propias, donde las herramientas generales no alcanzan.

Wireshark

- Captura y analiza el tráfico de red paquete por paquete.
- Permite verificar protocolos, tiempos y errores de comunicación.
- Se usa para diagnosticar problemas que las pruebas funcionales no explican.

Segron

- Automatiza pruebas de servicios de telecomunicaciones (redes móviles, señalización).
- Simula equipos y escenarios de red difíciles de reproducir con hardware real.
- Ejemplo de herramienta de dominio: solo tiene sentido en ese sector.

No todo el ecosistema es genérico: algunos dominios (redes, telecom, embebidos) necesitan herramientas propias.

Mapa resumen del ecosistema

Categoría	Qué resuelve	Herramientas de ejemplo
Gestión de pruebas	Planificar, organizar y trazar	TestRail, TestLink, Xray, JIRA+Zephyr
Análisis estático	Revisar el código sin ejecutarlo	SonarQube
Diseño / BDD	Especificar el comportamiento esperado	Cucumber
Ejecución / registro	Automatizar y correr pruebas	Selenium, Cypress, Appium, JUnit
Rendimiento	Medir carga, estrés y escalabilidad	JMeter, Gatling, BlazeMeter

Guardá esta tabla: es la que vamos a usar en la actividad que sigue.

El caso: ReservaLab

Una única regla de negocio, puesta a prueba desde todos los ángulos del ecosistema.

Política: cancelar hasta 24 horas antes del ingreso

CP-01

Faltan 10 h

Rechazo y explicación del plazo

CP-02

Faltan 30 h

Reserva cancelada

CP-03

Faltan 24 h (límite)

Definir criterio exacto y verificarlo

Un casillero "Simular defecto" fuerza el rechazo también de CP-02: así cada herramienta detecta el mismo problema desde su propio ángulo (UI, API, código, carga, red).

SonarQube

ANÁLISIS ESTÁTICO

Analiza el código fuente sin ejecutarlo. Puede bloquear un Quality Gate si detecta código riesgoso, y mostrar cobertura si se le importa un reporte.

En ReservaLab

Quality Gate	**NO SUPERADO**
Condición de la regla de cancelación	**Revisión requerida**
Cobertura importada de pruebas	**68 %**

Aporta a: evaluación objetiva del código, integrada al pipeline de CI/CD (Unidad 10).

Riesgo a vigilar: exceso de confianza — un Quality Gate en verde no reemplaza las pruebas dinámicas.